

1D , 2D And 3D Arrays

Arrays are data structures that store collections of elements of the same type in contiguous memory locations. They are classified by their dimensions, which dictate how data is indexed and organized: 1D (linear list), 2D (grid or matrix), and 3D (a "cube" of multiple 2D layers). [\[1, 2, 3\]](#)

1D (One-Dimensional) Array

A 1D array is the simplest form of array, functioning as a single linear sequence or list of elements. [\[1\]](#)

- Structure: A single row/column where each item is accessed by a single index.
- Visualization: Like a checklist or a single track of shelves.
- Common Use Cases: Storing a list of scores, a queue of tasks, or sequential logs.
- Syntax (C-like): `int arr[5];` (Creates a list that holds 5 integers). [\[1, 2, 3, 4, 5\]](#)

2D (Two-Dimensional) Array

A 2D array is essentially an array of 1D arrays, creating a table structured in rows and columns. [\[1, 2\]](#)

- Structure: Requires two indices to access an element, typically represented as `\(arr[row][column]\)`.
- Visualization: Like a chessboard, a spreadsheet, or a matrix.
- Common Use Cases: Storing image pixels, solving grid-based algorithms (like Tic-Tac-Toe), or tracking table-based data.
- Syntax (C-like): `int matrix[3][4];` (Creates a grid with 3 rows and 4 columns). [\[1, 2, 3, 4, 5\]](#)

3D (Three-Dimensional) Array

A 3D array is an array of 2D arrays, expanding the concept of rows and columns by adding a "depth" or "layer" axis. [\[1\]](#)

- Structure: Requires three indices to locate an item, represented as `\(arr[layer][row][column]\)`.

- Visualization: Like a Rubik's cube, a stack of multiple 2D spreadsheets, or 3D spatial coordinates.
- Common Use Cases: Representing multi-spectral image data, 3D gaming spaces, or OLAP (Online Analytical Processing) cubes in databases.
- Syntax (C-like): `int cube[2][3][4];` (Creates 2 layers of 3-row-by-4-column grids). [[1](#), [2](#), [3](#), [4](#), [5](#)]

Key Differences at a Glance

Feature [1, 2, 3, 4, 5]	1D Array	2D Array	3D Array
Dimensions	1 (Length)	2 (Row, Column)	3 (Layer, Row, Column)
Indexing	<code>arr[i]</code>	<code>arr[i][j]</code>	<code>arr[i][j][k]</code>
Memory Flow	Contiguous flat sequence	Row-major or column-major	Contiguous block of 2D arrays